Mohsin Iban Hossain

AIUB, Python Notes

PYTHON

Table of Contents

Sl. No	Topic	Pages
1	Introduction	03-12
2	Strings	13-19
3	Conditional Statements	20-23
4	Loops	24-26
5	Break & Continue	27-29
6	Lists	30-36
7	Tuples	37-39
8	Dictionary	40-47
9	Set	48-54
10	Functions	55-58
11	Recursion	59-60
12	Object-Oriented Programming (OOP)	61-76
13	File I/O	77-81
14	import in Python	82-83
15	Virtual Environment	84-87

INTRODUCTION TO PYTHON

Introduction

What is Python?

➔ **Python** is a high-level, easy-to-read programming language used for web development, data science, automation, and more. It is beginner-friendly, interpreted, and supports multiple programming styles.

➢ Our First Program in python:

Code:	Output:
`print("Hello World")`	Hello World

➢ Python Character Set:

➔ The **character set** in Python refers to all valid characters that can be used in writing Python programs.

1. Letters

- **Uppercase: `A` to `Z`**
- **Lowercase: `a` to `z`**

2. Digits

- **`0` to `9`**

3. Special Symbols

- `+ - * / % = < > ! & | ^ ~`
- `@ # $ _ ? : ; , . ' " \ ( ) [ ] { }`

4. Whitespace Characters

- **Space**
- **Tab (`\t`)**
- **Newline (`\n`)**
- **Carriage return (`\r`)**

Variables

➔ **Variable** is a name used to **store data** in memory. It acts as a container for values.

How to Declare a Variable:

Syntax:	**Note:**
x = 10 name = "Alice"	No need to declare the **type**—Python figures it out.
Rules for Naming Variables:	**Example Code:**

• Must start with a letter or underscore (_)
• Cannot start with a digit
• Can contain letters, digits, and underscores
• Case-sensitive (`age` and `Age` are different)
	`a = 5 # Integer`
	`pi = 3.14 # Float`
	`is_valid = True # Boolean`
	`name = "Mohsin" # String`

Data Types

➔ In Python, **data types** define the kind of value a variable hold.

Basic Data Types:

Type	Example	Description
`int`	`x = 10`	Whole numbers
`float`	`pi = 3.14`	Decimal numbers
`str`	`name = "Mohsin"`	Text (string of characters)
`bool`	`is_valid = True`	Boolean (True or False)

How to See the Data Type in Python:

Syntax:

```
type(variable)
```

Example Code:	Output:

`x = 10`
`print(type(x))  # <class 'int'>`
 	
`name = "Mohsin"`
`print(type(name))  # <class 'str'>`
 	
`price = 3.99`
`print(type(price))  # <class 'float'>`
 	
`is_valid = True`
`print(type(is_valid))  # <class 'bool'>`
	<class 'int'>
	<class 'str'>
	<class 'float'>
	<class 'bool'>

Keywords

➔ **Keywords** are reserved words in Python. We **cannot** use them as variable names or identifiers.

List of Common Python Keywords:

False	class	finally	is	return	None
continue	for	lambda	try	True	def
from	nonlocal	while	and	del	global
not	with	as	elif	if	or
yield	assert	else	import	pass	break
except	in	raise	async	await	

Example Code:

Correct:	Wrong:
`if True:` `  print("This is a keyword example.")`	`class = "Math"` `# ❌ Error! 'class' is a keyword`

- **Problem1:** Write a Python program that assign values to variables num1, num2 and perform addition operations. Also print the results.

Code:	Output:

`num1 = 10`
`num2 = 5`
`sum = num1+num2`
 	
`print("Sum:", sum)`
	Sum: 15

- **Problem2:** Write a Python program that assign values to variables num1, num2 and perform subtraction operations. Also print the results.

Code:	Output:

`num1 = 10`
`num2 = 5`
`sub = num1-num2`
 	
`print("Sub:", sub)`
	Sub: 5

- **Problem3:** Write a Python program that assign values to variables num1, num2 and perform multiplication & division operations. Also print the results.

Code:	Output:

`num1 = 10`
`num2 = 5`
`mult = num1*num2`
`div = num1/num2`
 	
`print("Mult:",mult)`
`print("Div:",div)`
	Mult: 50
	Div: 2.0

Comments in Python

➔ **Comments** are notes in the code that are ignored by Python during execution. They help explain the code for humans.

Types of Comments:

Single-line Comment: starts with #	**Multi-line Comment:** use triple quotes ''' ''' or """ """

`# This is a single-line comment`
`print("Hello")  # This prints Hello`
	`'''`
	`This is a`
	`multi-line comment`
	`'''`
	`print("Hi")`

Note: Triple quotes are technically multi-line strings, but often used as multi-line comments.	

Types of Operators

➔ Python supports **several types of operators** to perform operations on variables and values.

Type	Example Operators	Description
1. Arithmetic	`+ - * / % // **`	Used for math operations
2. Assignment	`= += -= *= /=`	Assign or update values
3. Comparison	`== != > < >= <=`	Compare values (returns `True`/`False`)
4. Logical	`and or not`	Combine conditions
5. Bitwise	`&`	^ ~ << >>`

Example Code:

Types of Operators:

```
x = 10              # Assignment
y = 5               # Assignment

print(x + y)        # Arithmetic
print(x > y)        # Comparison
print(x != y)       # Comparison
print(x and y)      # Logical
```

Type Conversion

➔ **Type Conversion** means changing the data type of a value or variable.

Types of Type Conversion:

Implicit Conversion (Automatic):

➔Python automatically converts one data type to another when needed.

```
x = 10        # int
y = 2.5       # float
result = x + y
print(result)       # 12.5 (float)
print(type(result)) # <class 'float'>
```

Explicit Conversion (Type Casting):

➔We manually convert types using built-in functions:

Syntax:

```
int()    # Converts to integer
float()  # Converts to float
str()    # Converts to string
bool()   # Converts to boolean
```

Example code:	Output:

`x = 5.9`
`print(int(x))  # 5`
`print(type(x))  # <class 'float'>`
 	
`y = 100`
`print(str(y))  # "100"`
`print(type(y))  # <class 'int'>`
 	
`z = "45"`
`print(int(z))  # 45`
`print(type(z))  #<class 'str'>`
	`5`
	`<class 'float'>`
	
	`100`
	`<class 'int'>`
	
	`45`
	`<class 'str'>`

Input in Python

➔ The `input()` function is used to **take user input** in Python.

Syntax:

```
variable = input("Enter something: ")
```

Example Code:

Example code:

```
name = input("Enter your name: ")
print("Hello,", name)
```

Output:

```
Enter your name: Mohsin Ibna Hossain
Hello, Mohsin Ibna Hossain
```

Convert input to other types:

```
# Converts input to integer
age = int(input("Enter your age: "))

# Converts input to float
price = float(input("Enter price: "))

print("Age:",age)
print("Price:",price)
```

Output:

```
Enter your age: 23
Enter price: 99.99
Age: 23
Price: 99.99
```

Practice Problem

ø **Problem1:** Write a Program to input 2 numbers & print their sum.

Code:

```
num1 = int(input("Enter number 1: "))
num2 = int(input("Enter number 2: "))
sum = num1+num2

print("Sum:",sum)
```

Output:

```
Enter number 1: 10
Enter number 2: 20
Sum: 30
```

- **Problem2:** Write a Python program to take five float variables as input from the user. Find out the summation and average of five numbers

Code:	Output:

```
num1 = float(input("Enter number 1: "))
num2 = float(input("Enter number 2: "))
num3 = float(input("Enter number 3: "))
num4 = float(input("Enter number 4: "))
num5 = float(input("Enter number 5: "))

summ = num1 + num2 + num3 + num4 + num5
average = summ / 5

print("Summation:", summ)
print("Average:", average)
```

```
Enter number 1: 10
Enter number 2: 3
Enter number 3: 7
Enter number 4: 6
Enter number 5: 8
Summation: 34.0
Average: 6.8
```

- **Problem3:** Write a python program that takes an integer variable, Z and computes the result of $Z^5 + Z^7 + Z^9 + Z^4 - Z^3 * Z^2$. Do not use any built-in function.

Code:	Output:

```
z = int(input("Enter an integer number: "))

z2 = z * z       # Z^2
z3 = z2 * z      # Z^3
z4 = z2 * z2     # Z^4
z5 = z4 * z      # Z^5
z7 = z4 * z3     # Z^7
z9 = z5 * z4     # Z^9

result = z5 + z7 + z9 + z4 - (z3 * z2)

print("Z⁵ + Z⁷ + Z⁹ + Z⁴ - Z³*Z² =", result)
```

```
Enter an integer number: 4
Z⁵ + Z⁷ + Z⁹ + Z⁴ - Z³*Z² = 278784
```

- **Problem4:** Write a python program that has two variables, Base and Height and it computes the area of a triangle.

Code:	Output:

`a=float(input("Enter Base: "))`
`b=float(input("Enter Height: "))`
 	
`area=0.5*a*b`
 	
`print("Area is:",area)`
	`Enter Base: 5`
	`Enter Height: 4`
	`Area is: 10.0`

- **Problem5:** Write a Python program to input two integer numbers, a and b. The program should print True if a is greater than or equal to b. Otherwise, print False.

Code:	Output:

`a = int(input("Enter the integer (a): "))`
`b = int(input("Enter the integer (b): "))`
 	
`result = a >= b`
 	
`print(result)`
	`Enter the integer (a): 10`
	`Enter the integer (b): 8`
	`True`

	`Enter the integer (a): 10`
	`Enter the integer (b): 20`
	`False`

- **Problem6:** Write a python program that has one variables, Side and it computes the area of a square.

Code:	Output:

`side = float(input("Enter side: "))`
`area = side*side`
 	
`print("Area is:",area)`
	`Enter side: 5`
	`Area is: 25.0`


STRING

Strings

➔ A **string** is a sequence of characters enclosed in **single**, **double**, or **triple quotes**.

Common String Operations:

Operation	Example	Output
Concatenation	`"Hello" + " World"`	`'Hello World'`
Repetition	`"Hi" * 3`	`'HiHiHi'`
Indexing	`"Hello"[1]`	`'e'`
Slicing	`"Hello"[1:4]`	`'ell'`
Length	`len("Hello")`	`5`
Uppercase	`"hello".upper()`	`'HELLO'`
Lowercase	`"HELLO".lower()`	`'hello'`
Replace	`"apple".replace("a", "A")`	`'Apple'`
Strip	`" hello ".strip()`	`'hello'`

Concatenation

➔ **Concatenation** means **joining two or more strings** together using the + operator.

Example code:	Output:

`str1 = "Hello"`
`str2 = "World"`
`result = str1 + " " + str2`
`print(result)`
	`Hello World`

Repetition

➔ **Repetition** means repeating a string multiple times using the * operator

Example code:	Output:

`text = "Hi"`
`result = text * 3`
`print(result)`
	`HiHiHi`

Note: Works only with string and integer	

Indexing

➔ **Indexing** is used to access individual characters in a string using their position (index).

Example code:	Output:

`text = "Python"`
`print(text[0])  # Output: P`
`print(text[3])  # Output: h`
	P
	h

Note: Indexing starts at 0

IndexError: Trying to access an index that doesn't exist will raise an error

```
text = "Python"
print(text[10])  # ❌ IndexError
```

Negative indexing: Negative indexing starts from the end

Example code:	Output:

`text = "Python"`
`print(text[-1])  # Output: n (last character)`
`print(text[-3])  # Output: h`
	n
	h

Slicing

➔ **Slicing** allows you to extract a **portion of a string** using a range of indices.

Syntax:

```
string[start : end]
```

Example code:	Output:

`text = "Python"`
`print(text[1:4])  # Output: yth`
`print(text[:3])  # Output: Pyt`
`print(text[2:])  # Output: thon`
`print(text[-4:-1]) # Output: tho`
	yth
	Pyt
	thon
	tho

Note: Includes characters from **start index** up to, but **not including, end.**

Length

➔ To find the **length** (number of characters) in a string, use the built-in `len()` function.

Syntax:

```
len(string)
```

Example code:	Output:
`msg = "Hi there!"` `print(len(msg))  # Output: 9`	**9**

Note: Counts **all characters**, including spaces, symbols, and numbers.

Uppercase

➔ To convert all characters in a string to **uppercase**, use the `.upper()` method.

Syntax:

```
string.upper()
```

Example code:	Output:
`text = "python"` `print(text.upper())  # Output: PYTHON`	**PYTHON**

Notes:

- Doesn't change the original string (strings are immutable).
- Returns a **new string** with all letters in uppercase.

Lowercase

➔ To convert all characters in a string to **lowercase**, use the `.lower()` method.

Syntax:

```
string.lower()
```

Example code:	Output:
`text = "HELLO"` `print(text.lower())  # Output: hello`	**hello**

Notes:

- Returns a new string in lowercase.
- Useful for case-insensitive comparisons.

Replace

➔ The `.replace()` method is used to **replace parts of a string** with another string.

Syntax:

```
string.replace(old, new)
```

Example code:	Output:

`text = "I like apples"`
`print(text.replace("apples", "oranges"))`
`# Output: I like oranges`
	I like oranges

Notes:

- It does not change the original string.
- We can chain **.replace()** multiple times if needed.

Strip

➔ The `.strip()` method is used to **remove whitespace or specified characters** from the **start and end** of a string.

Syntax:

```
string.strip()              # removes spaces
string.strip(chars)         # removes specified characters
```

Example code:

```
text = "  Hello  "
print(text)
print(text.strip())

name = "***Mohsin***"
print(name)
print(name.strip("*"))
```

Output:

```
  Hello
Hello

***Mohsin***
Mohsin
```

Notes:

- Use **.lstrip()** to remove from the left only.
- Use **.rstrip()** to remove from the right only.

String Functions

String Functions:

```
str1 = "I am a coder."

print(str1.endswith("er."))          # returns True if string ends with "er."
print(str1.capitalize())             # capitalizes the first character
print(str1.find("a"))                # returns index of first occurrence of "a"
print(str1.count("am"))              # counts the occurrence of "am"
```

Practice Problem

- **Problem1:** Write a Program to input user's first name & print its length.

Code:	Output:
``` first_name = input("Enter first name: ") length = len(first_name) print("Length of first name is:", length) ```	``` Enter first name: Mohsin Length of first name is: 6 ```

- **Problem2:** Write a Python program to initialize a string and find how many times the character '$' appears in it. text = "He bought a pen for $5 and a book for $10, total $15."

Code:	Output:
``` text = "He bought a pen for $5 and a book for $10, total $15." count = text.count('$') print("The character '$' appears", count, "time(s).") ```	``` The character '$' appears 3 time(s). ```

- **Problem3:** Write a Python program to initialize a string containing a sentence, and then convert and print the entire string in **uppercase letters** using a string function.

Code:	Output:
``` sentence = "Learning Python is fun." upper_sentence = sentence.upper() print("Uppercase:", upper_sentence) ```	``` Uppercase: LEARNING PYTHON IS FUN ```

CONDITIONAL STATEMENTS

Conditional Statements

➔ **Conditional statements** let you execute different blocks of code based on conditions.

Types of Conditional Statements:

`if` Statement:

```
x = 10
if x > 0:
    print("Positive number")
```

`if...else` Statement:

```
x = -5
if x >= 0:
    print("Positive")
else:
    print("Negative")
```

`if...elif...else` Statement:

```
x = 0
if x > 0:
    print("Positive")
elif x == 0:
    print("Zero")
else:
    print("Negative")
```

Notes:

- Conditions use **comparison operators** (==, !=, >, <, >=, <=)
- Python uses **indentation** to define blocks

Nested Conditional Statements

➔ A **nested conditional** means using one `if` or `else` block **inside another**.
It allows for **more complex decision making**.

Syntax:

```
if condition1:
    if condition2:
        # Block 1
    else:
        # Block 2
else:
    # Block 3
```

Example code:

```
age = 20
has_id = True

if age >= 18:
    if has_id:
        print("You are allowed to enter.")
    else:
        print("Please show your ID.")
else:
    print("You are not allowed to enter.")
```

Output:

```
You are allowed to enter.
```

Notes:

- Use indentation correctly (4 spaces recommended)
- Helps when multiple levels of checking are required
- Can combine with elif if needed

Practice Problem

- **Problem1:** Write a Python program to check whether a number entered by the user is **odd** or **even**, and print the result.

Code:	Output:

`num = int(input("Enter a number: "))`
 	
`if num % 2 == 0:`
`    print("Even")`
`else:`
`    print("Odd")`
	`Enter a number: 10`
	`Even`
	
	`Enter a number: 5`
	`Odd`

- **Problem2:** Write a Python program to **grade students based on their marks** using the following criteria:

- If marks are **90 or above**, grade = "A"
- If marks are **80 to 89**, grade = "B"
- If marks are **70 to 79**, grade = "C"
- If marks are **below 70**, grade = "D"

Then print the grade.

Code:	Output:

`marks = int(input("Enter student's marks: "))`
 	
`if marks >= 90:`
`    grade = "A"`
`elif marks >= 80:`
`    grade = "B"`
`elif marks >= 70:`
`    grade = "C"`
`else:`
`    grade = "D"`
 	
`print("Grade:", grade)`
	`Enter student's marks: 73`
	`Grade: C`

- **Problem3:** Write a Python program to input three numbers from the user and print the **greatest** among them.

Code:

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
c = int(input("Enter third number: "))

if a >= b and a >= c:
    greatest = a
elif b >= a and b >= c:
    greatest = b
else:
    greatest = c

print("The greatest number is:", greatest)
```

Output:

```
Enter first number: 10
Enter second number: 20
Enter third number: 30
The greatest number is: 30
```

```
Enter first number: 15
Enter second number: 10
Enter third number: 5
The greatest number is: 15
```

```
Enter first number: 7
Enter second number: 21
Enter third number: 14
The greatest number is: 21
```

- **Problem6: Write a Python program to check whether a number is divisible by 2, 5, and 11.** Also, **demonstrate the use of both OR and AND operators** in the program.

Code:

```
num = int(input("Enter a number: "))

# Using AND operator to check if divisible by 2, 5, and 11
if num % 2 == 0 and num % 5 == 0 and num % 11 == 0:
    print("number is divisible by 2, 5, and 11.")
else:
    print("number is NOT divisible by 2, 5, and 11.")

# Optional: Demonstrating OR operator (for learning)
if num % 2 == 0 or num % 5 == 0 or num % 11 == 0:
    print("number is divisible by at least one of 2, 5, or 11.")
else:
    print("number is not divisible by 2, 5, or 11.")
```

Output:

```
Enter a number: 110
number is divisible by 2, 5, and 11.
number is divisible by at least one of 2, 5, or 11.
```

```
Enter a number: 37
number is NOT divisible by 2, 5, and 11.
number is not divisible by 2, 5, or 11.
```

Loops

➔ **Loops** are used to execute a block of code repeatedly.

Types of Loops:

`for` Loop: iterate over a sequence	`while` Loop: runs while a condition is `True`

`for i in range(0?,5,2?):`
`  print(i)  # Output: 0 2 4`
	`count = 0`
	`while count < 3:`
	`  print(count)  # Output: 0 1 2`
	`  count += 1`

Notes: `range(Start?, End, Step?)` functions returns a sequence of numbers, starting from start (**0 by default),** and **increments by Step (1 by default),** and stops before a specified number.	

Practice Problem

- **Problem1:** Write a Python program to **print numbers from 1 to 5 using a `while` loop**.

Code:	Output:

`i = 1`
`while i <= 5:`
`  print(i)`
`  i += 1`
	1
	2
	3
	4
	5

- **Problem2:** Write a Python program to **print numbers from 5 to 1** using a while loop.

Code:	Output:

`i = 5`
`while i >= 1:`
`  print(i)`
`  i -= 1`
	5
	4
	3
	2
	1

- **Problem3:** Write a Python program to **input a number n** and print the **sum of all numbers from 1 to n** using a while loop.

Code:	Output:

`n = int(input("Enter a number: "))`
 	
`i = 1`
`total = 0`
 	
`while i <= n:`
`  total += i`
`  i += 1`
 	
`print("Sum from 1 to", n, "is:", total)`
	`Enter a number: 5`
	`Sum from 1 to 5 is: 15`

- **Problem4:** Write a Python program to input a number n and print its **multiplication table** using a while loop.

Code:	Output:

``
n = int(input("Enter a number: "))
 	
i = 1
while i <= 10:
    print(n, "x", i, "=", n * i)
    i += 1
``
	Enter a number: 11
	11 x 1 = 11
	11 x 2 = 22
	11 x 3 = 33
	11 x 4 = 44
	11 x 5 = 55
	11 x 6 = 66
	11 x 7 = 77
	11 x 8 = 88
	11 x 9 = 99
	11 x 10 = 110

- **Problem5:** Write a Python program to **print numbers from 1 to 5 using a `using for` loop `& range().`**

Code:	Output:

``
for i in range(1, 6):
    print(i)
``
	1
	2
	3
	4
	5

- **Problem6:** Write a Python program to **print numbers from 5 to 1 using a `using for` loop `& range().`**

Code:	Output:

``
for i in range(5, 0, -1):
    print(i)
``
	5
	4
	3
	2
	1

- **Problem7:** Write a Python program to **input a number n** and print the **sum of all numbers from 1 to n** using a `for` loop `& range().`

Code:	Output:

``
n = int(input("Enter a number: "))
total = 0
 	
for i in range(1, n + 1):
    total += i
 	
print("Sum from 1 to", n, "is:", total)
``
	Enter a number: 5
	Sum from 1 to 5 is: 15

BREAK & CONTINUE

Break & Continue

➔ Used to **control the flow** inside loops.

`break` Statement:	`continue` Statement:

`for i in range(5):`
`  if i == 3:`
`  break`
`  print(i)  # Output: 0 1 2`
	`for i in range(5):`
	`  if i == 3:`
	`  continue`
	`  print(i)  # Output: 0 1 2 4`

Note: **Stops** the loop immediately.	**Note:** **Skips** the current iteration and moves to the next.

Pass Statement

➔ The `pass` statement is a **placeholder** used when a statement is required syntactically, but no code needs to run.

Syntax:	**Example:**

`if True:`
`  pass  # Do nothing`
	`for i in range(5):`
	`  if i == 3:`
	`  pass  # Placeholder, does nothing`
	`  print(i)`

Use Cases: • Empty function or class definitions • Placeholder for future code • Avoid syntax errors when a block is needed	

Printing Output in a Single Line

➔ By default, `print()` adds a **newline** (`\n`) after each call. To print on the **same line**, use the `end` parameter.

Syntax:

```
print(value, end=" ")
```

Example code:	Output:
`for i in range(3):` `  print(i, end=" ")`	0 1 2

Notes:

- end=" " adds a space instead of a new line
- You can use any character: end="-", end="", etc.

Practice Problem

- **Problem1:** Write a Python program that takes numbers as input from the user using a `while` loop. The loop should **stop if the user enters** `-1`. Print the **sum** of all entered numbers (excluding `-1`).

Code:	Output:

`total = 0`
 	
`while True:`
`  num = int(input("Enter a number (-1 to stop): "))`
`  if num == -1:`
`  break`
`  total += num`
 	
`print("Total sum is:", total)`
	Enter a number (-1 to stop): 10
	Enter a number (-1 to stop): 5
	Enter a number (-1 to stop): 3
	Enter a number (-1 to stop): 2
	Enter a number (-1 to stop): 1
	Enter a number (-1 to stop): -1
	Total sum is: 21

- **Problem2:** Write a Python program to print all numbers from **1 to 10 in a line**, except for number **5**, using a `for` loop and the `continue` statement.

Code:	Output:

`for i in range(1, 11):`
`  if i == 5:`
`  continue`
`  print(i , end=" ")`
	1 2 3 4 6 7 8 9 10

LISTS

Lists

➔ A **list** is an **ordered**, **changeable (mutable)** collection that allows **duplicate** values. Just like **Array.**

Syntax:	Example:
`llistVariableName = ["item1", "item2"]`	`fruits = ["apple", "banana", "cherry"]`

Key Features:

- Can contain **mixed data types** (e.g., numbers, strings, etc.)

Common List Operations:

Operation	Example	Description
Access item	`fruits[0]`	`'apple'`
Change item	`fruits[1] = "mango"`	Replaces `'banana'` with `'mango'`
Add item	`fruits.append("orange")`	Adds to end
Insert item	`fruits.insert(1, "kiwi")`	Inserts at index 1
Remove item	`fruits.remove("apple")`	Removes `'apple'`
Pop item	`fruits.pop()`	Removes last item
Length	`len(fruits)`	Number of items
Sort	`fruits.sort()`	Sorts list
Reverse	`fruits.reverse()`	Reverses list
Slicing	`fruits[1:3]`	`'banana', 'cherry'`

Access item

➔ We can access elements in a list using **indexing**.

Syntax:

```
list_name[index]   #Indexing starts at 0 (first item), negative indexing starts from the end.
```

Example code:	Output:

`fruits = ["apple", "banana", "cherry"]`
 	
`print(fruits[0])  # Output: apple`
`print(fruits[1])  # Output: banana`
`print(fruits[-1])  # Output: cherry`
	apple
	banana
	cherry

Change item

➔ Lists are **mutable**, so you can **change the value** of an item by referring to its index.

Syntax:

```
list_name[index] = new_value
```

Example code:

```
fruits = ["apple", "banana", "cherry"]
print("Before:",fruits)

fruits[1] = "mango"
print("After:",fruits)
```

Output:

```
Before: ['apple', 'banana', 'cherry']
After: ['apple', 'mango', 'cherry']
```

Notes:

- Index **must be valid** (within range)
- Only one item is updated at a time

Add item

➔ You can add items to a list using `append()` or `insert()` methods.

`extend()` – Add multiple items:

```
fruits = ["apple", "banana"]
fruits.extend(["grape", "melon"])

print(fruits)
# Output: ['apple', 'banana', 'grape', 'melon']
```

`append()` – Add at the end:

```
fruits = ["apple", "banana"]
fruits.append("cherry")

print(fruits)
# Output: ['apple', 'banana', 'cherry']
```

`insert(index, item)` – Add at specific position:

```
fruits = ["apple", "banana"]
fruits.insert(1, "orange")

print(fruits)
# Output: ['apple', 'orange', 'banana']
```

Inserting item

➔ To add an item at a **specific position** in a list, use the **insert()** method.

Syntax:

```
list_name.insert(index, item)
```

Example code:

```
fruits = ["apple", "banana", "cherry"]
print("Before:",fruits)

fruits.insert(1, "orange")
print("After:",fruits)
```

Output:

```
Before: ['apple', 'banana', 'cherry']
After: ['apple', 'orange', 'banana', 'cherry']
```

Notes:

- Index starts at 0
- Shifts elements to the right
- If index is greater than the length, item is added at the end

Remove item

➔ We can **remove** items from a list using the `remove()` method.

Syntax:

```
list_name.remove(item)
```

Example code:

```
fruits = ["apple", "banana", "cherry"]
fruits.remove("banana")
print(fruits)
```

Output:

```
['apple', 'cherry']
```

Notes:

- Removes first **occurrence** of the specified item.
- If the item is **not found**, it raises a **ValueError**.

Pop item

➔ The **pop()** method removes an item from the list **by index** and returns it.

Syntax:

```
list_name.pop(index)  # index is optional
```

Example code:	Output:

`fruits = ["apple", "banana", "cherry"]`
`fruits.pop()`
 	
`print(fruits)`
	`['apple', 'banana']`

Notes:

- If index is out of range, it raises an IndexError.
- Unlike remove(), pop() can be used to get the removed value.

List Methods

List Methods:

```
list = [2, 1, 3]

list.append(4)  #adds one element at the end [2, 1, 3,4]
list.sort( )  #sorts in ascending order [1, 2, 3]
list.sort( reverse=True )  #sorts in descending order [3, 2, 1]
list.reverse( )  #reverses list [3, 1, 2]
```

Practice Problem

- **Problem1:** Write a Python program to **ask the user to enter the names of their 3 favorite movies** and **store them in a list**.

Code:	Output:

``
movies = []
 	
for i in range(3):
    movie = input("Enter the name of movie: ")
    movies.append(movie)
 	
print("Your favorite movies are:\n", movies)
``
	Enter the name of movie: Inception
	Enter the name of movie: Interstellar
	Enter the name of movie: The Matrix
	Your favorite movies are:
	['Inception', 'Interstellar', 'Matrix']

- **Problem2:** Write a Python program to **check if a list is a palindrome**, meaning the list reads the same forward and backward. **(Hint: Use the copy() method)**

Code:	Output:

``
elements = input("Enter elements: ").split()
 	
original_list = elements.copy()
original_list.reverse()
reversed_list = original_list
 	
if elements == reversed_list:
    print("The list is a palindrome.")
else:
    print("The list is not a palindrome.")
``
	Enter elements: 12121
	The list is a palindrome.
	
	Enter elements: 1212
	The list is a palindrome.

Taking User Input in a List

➔ We can take list input from a user using a loop or by splitting a string input.

Method 1: Using `split()` (for space-separated input)

Example code:

```
names = input("Enter Name: ").split()
print(names)
```

Output:

```
Enter Name: JAM, MOHSIN, NIAHN, ALIF
['JAM,', 'MOHSIN,', 'NIAHN,', 'ALIF']
```

Method 2: Using a loop (for multiple inputs)

Example code:

```
n = int(input("How many items? "))
items = []
for i in range(n):
    item = input(f"Enter item {i+1}: ")
    items.append(item)

print(items)
```

Output:

```
Enter item 1: apple
Enter item 2: banana
Enter item 3: cherry
['apple', 'banana', 'cherry']
```

Notes: `input(f"Enter item {i+1}: ")` uses an **f-string** to format the prompt correctly, displaying the item number to the user.

Practice Problem

- **Problem1:** Write a Python program that takes **5 numbers as input from the user**, **stores them in a list**, and then **prints the list** along with the **sum of all the numbers**.

Code:

```
numbers = []
sum = 0

for i in range(5):
    num = int(input("Enter a number: "))
    numbers.append(num)
    sum +=num

print("Your list:", numbers)
print("Sum of all numbers:",sum)
```

Output:

```
Enter a number: 10
Enter a number: 20
Enter a number: 30
Enter a number: 40
Enter a number: 50
Your list: [10, 20, 30, 40, 50]
Sum of all numbers: 150
```

TUPLES

Tuples

➔ A **tuple** is an **ordered**, **immutable** (unchangeable) collection of items. It is similar to a list but cannot be modified after creation.

Syntax:	Example:
`my_tuple = (item1, item2, item3)`	`fruits = ("apple", "banana", "cherry")` `print(fruits[0])  # Output: apple`

Key Features:

- Defined using **parentheses** `()`
- **Immutable**: We cannot change, add, or remove items after creation
- Allows **duplicate** values
- Can contain **mixed data types** (e.g., numbers, strings, etc.)

Tuple with One Item:

Example code:	Output:

`one_item = ("apple",)`
`print(one_item)`
`print(type(one_item))`
 	
`one_item = ("apple")`
`print(one_item)`
`print(type(one_item))`
	`('apple',)`
	`<class 'tuple'>`
	
	`apple`
	`<class 'str'>`

Notes: Must include comma!

Tuples Methods

Tuples Methods:

```
fruits = ("apple", "banana", "cherry")

len(fruits)              # Length of the tuple
fruits.index("banana")   # Find index of an item
fruits.count("apple")    # Count how many times an item appears
```

Practice Problem

Problem1: **Write a Python program to count how many students received an `"A"` grade** in the following **list**: ["C", "D", "A", "A", "B", "B", "A"]

Code:	Output:

`grades = ("C", "D", "A", "A", "B", "B", "A")`
 	
`count_A = grades.count("A")`
`print("Number of 'A' grade:", count_A)`
	Number of 'A' grade: 3

Problem2: Write a Python program to **input 5 numbers from the user**, store them in a **tuple**, and then **print the maximum and minimum values**.

Code:	Output:

`numbers = []`
 	
`for i in range(5):`
`  num = int(input("Enter a number: "))`
`  numbers.append(num)`
 	
`number_tuple = tuple(numbers)`
 	
`print("Tuple:", number_tuple)`
`print("Maximum value:", max(number_tuple))`
`print("Minimum value:", min(number_tuple))`
	Enter a number: 10
	Enter a number: 20
	Enter a number: 30
	Enter a number: 40
	Enter a number: 50
	Tuple: (10, 20, 30, 40, 50)
	Maximum value: 50
	Minimum value: 10

Problem3: Create a tuple with **mixed data types** (e.g., `("Mohsin", 21, "CSE")`) and **print each item along with its data type**.

Code:	Output:

`info = ("Mohsin", 21, "CSE")`
 	
`for item in info:`
`  print(item, "=>", type(item))`
	Mohsin => <class 'str'>
	21 => <class 'int'>
	CSE => <class 'str'>

Dictionary

➔ A **dictionary** is an **unordered**, **mutable** collection that stores **key:value pairs**.

Syntax:

```
my_dict = {
    "key1": "value1",
    "key2": "value2"
}
```

Example:

```
student = {
    "name": "Mohsin",
    "age": 21,
    "department": "CSE"
}
print(student["name"])  # Output: Mohsin
```

Key Features:

- Keys must be **unique**
- Values can be **any data type**
- Defined using **curly braces {}**

Common Dictionary Methods:

Method	Description
`dict.keys()`	Returns all keys
`dict.values()`	Returns all values
`dict.items()`	Returns all key-value pairs
`dict.get("key")`	Gets value for key, returns `None` if absent
`dict["key"] = value`	Adds or updates a key-value pair
`dict.pop("key")`	Removes a key-value pair

dict.keys() Method

➔ The `.keys()` method returns a **view object** containing all the **keys** from a dictionary.

Syntax:

```
dictionary.keys()
```

Example code:

```
student = {
    "name": "Alice",
    "age": 21,
    "dept": "CSE"
}

print(student.keys())
```

Output

```
dict_keys(['name', 'age', 'dept'])
```

dict.values() Method

➔ The `.values()` method returns a **view object** containing all the **values** in the dictionary.

Syntax:

```
dictionary.values()
```

Example code:

```
student = {
    "name": "Alif",
    "age": 21,
    "dept": "CSE"
}

print(student.values())
```

Output:

```
dict_values(['Alif', 21, 'CSE'])
```

dict.get("key") Method

➔ The `.get()` method returns the **value** for the specified key. If the key is not found, it returns `None` (or a default value if provided), **without causing an error**.

Syntax:

```
dictionary.get("key", default_value?)
```

Example code:

```
student = {
    "name": "Nihan",
    "age": 21
}

print(student.get("name"))
print(student.get("dept"))
print(student.get("dept", "N/A"))
```

Output:

```
Nihan
None
N/A
```

Difference from `dict["key"]`:

- `student["dept"]` → ❌ Raises `KeyError` if key **doesn't exist**
- `student.get("dept")` → ✅ Returns `None` (or default)

dict.items() Method

➔ The `.items()` method returns a **view object** containing all **key-value pairs** as tuples.

Syntax:

```
dictionary.items()
```

Example code:

```
student = {
    "name": "Jam",
    "age": 21,
    "dept": "CSE"
}

print(student.items())
```

Output:

```
dict_items([('name', 'Jam'), ('age', 21), ('dept', 'CSE')])
```

dict["key"] = value

➔ The **dict["key"] = value** syntax is used to **add** a new key-value pair or **update** the value of an existing key in a dictionary.

Syntax:

```
dictionary["key"] = value
```

Example code:

```
student = {
    "name": "Mohsin",
    "age": 21
}
# Add new key-value
student["dept"] = "CSE"

# Update existing value
student["age"] = 22

print(student)
```

Output:

```
{'name': 'Mohsin', 'age': 22, 'dept': 'CSE'}
```

Use Case:

- If the key exists → updates the value
- If the key doesn't exist → adds the key-value pair

dict.pop("key") Method

➔ The `.pop("key")` method **removes** the specified key from the dictionary and **returns its value**.

Syntax:

```
dictionary.pop("key", default_value?)
```

Example code:	Output:

```
student = {
    "name": "Alice",
    "age": 21,
    "dept": "CSE"
}

removed_value = student.pop("age")
print(removed_value)
print(student)
```

```
21
{'name': 'Alice', 'dept': 'CSE'}
```

Nested Dictionary

➔ A **nested dictionary** is a dictionary **inside another dictionary**. It allows you to represent complex structured data.

Syntax:

```
nested_dict = {
    "key1": {
        "subkey1": value1,
        "subkey2": value2
    },
    "key2": {
        "subkey1": value1,
        "subkey2": value2
    }
}
```

Example code:	Output:
	

```
students = {
    "23-50194-1": {
        "name": "Mohsin",
        "age": 21
    },
    "23-50187-1": {
        "name": "Jam",
        "age": 22
    }
}

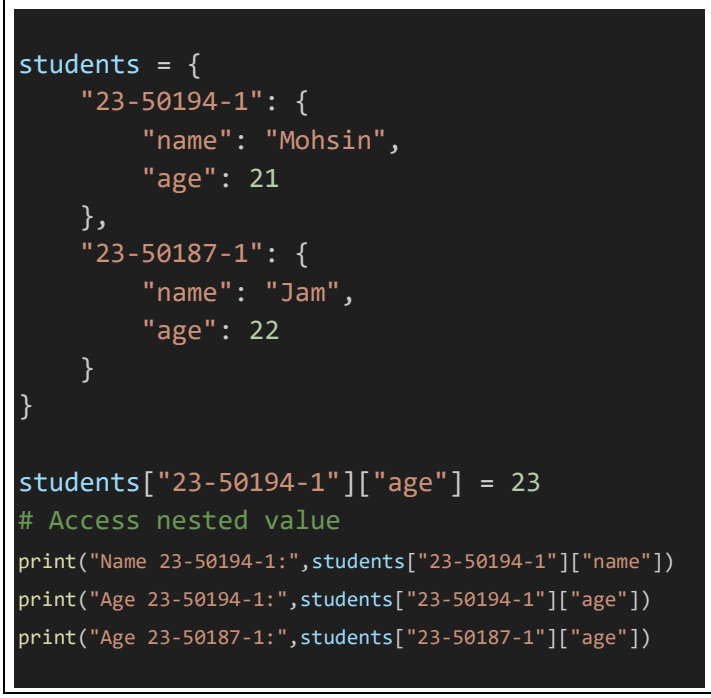
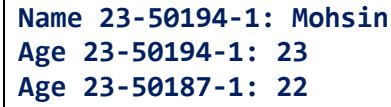
# Access nested value
print(students["23-50194-1"]["name"])
print(students["23-50187-1"]["age"])
```

```
Mohsin
22
```

Modify Nested Value

Syntax:

```
students["23-50194-1"]["age"] = 23
```

Example code:	Output:
	

```
students = {
    "23-50194-1": {
        "name": "Mohsin",
        "age": 21
    },
    "23-50187-1": {
        "name": "Jam",
        "age": 22
    }
}

students["23-50194-1"]["age"] = 23
# Access nested value
print("Name 23-50194-1:",students["23-50194-1"]["name"])
print("Age 23-50194-1:",students["23-50194-1"]["age"])
print("Age 23-50187-1:",students["23-50187-1"]["age"])
```

```
Name 23-50194-1: Mohsin
Age 23-50194-1: 23
Age 23-50187-1: 22
```

Add New Inner Dictionary

Syntax:

```
students["23-50200-1"] = {
    "name": "Nihan",
    "age": 23
}
```

Example code:

```
students = {
    "23-50194-1": {
        "name": "Mohsin",
        "age": 24
    },
    "23-50187-1": {
        "name": "Jam",
        "age": 22
    }
}

students["23-50200-1"] = {
    "name": "Nihan",
    "age": 23
}

students["23-50186-1"] = {
    "name": "Alif",
    "age": 21
}

# Access nested value
print("23-50186-1:",students["23-50186-1"])
print("23-50187-1:",students["23-50187-1"])
print("23-50194-1:",students["23-50194-1"])
print("23-50200-1:",students["23-50200-1"])
```

Output:

```
23-50186-1: {'name': 'Alif', 'age': 21}
23-50187-1: {'name': 'Jam', 'age': 22}
23-50194-1: {'name': 'Mohsin', 'age': 24}
23-50200-1: {'name': 'Nihan', 'age': 23}
```

Practice Problem

Problem1: Store the following word meanings in a **Python dictionary**, where each word can have **multiple meanings**.

`table → "a piece of furniture","list of facts & figures"`

`cat → "a small animal"`

Code:

```
dictionary = {
    "table": ["a piece of furniture", "list of facts & figures"],
    "cat": ["a small animal"]
}

# Printing the dictionary
print(dictionary)
print()

print("Table:",dictionary["table"])
print("Cat:",dictionary["table"])
```

Output:

```
{'table': ['a piece of furniture', 'list of facts & figures'], 'cat': ['a small animal']}

Table: ['a piece of furniture', 'list of facts & figures']
Cat: ['a piece of furniture', 'list of facts & figures']
```

Problem2: Write a Python program to **enter marks of 3 subjects** from the user and **store them in a dictionary**. Start with an **empty dictionary** and add subject–marks **one by one** using the subject name as the key and the marks as the value.

Code:

```
marks_dict = {}

for i in range(3):
    subject = input("Enter subject name: ")
    marks = int(input(f"Enter marks for {subject}: "))
    marks_dict[subject] = marks

print("Marks Dictionary:", marks_dict)
```

Output:

```
Enter subject name: Physics
Enter marks for Physics: 99
Enter subject name: English
Enter marks for English: 78
Enter subject name: Math
Enter marks for Math: 100
Marks Dictionary: {'Physics': 99, 'English': 78, 'Math': 100}
```


SET

Set

➔ A **Set** is an **unordered**, **unindexed** collection of **unique elements**. It is useful when we want to store items without duplicates.

Syntax:	Example:

`my_set = {1, 2, 3}`	`numbers = {1, 2, 3, 2, 1}`
	`print(numbers)`
	`# Output: {1, 2, 3} → duplicates are removed`

Common Set Methods:

Operation	Code Example	Result
Add item	`my_set.add(4)`	Adds 4 to the set
Remove item	`my_set.remove(2)`	Removes 2, raises error if not found
Discard item	`my_set.discard(2)`	Removes 2, **no error** if not found
Pop random item	`my_set.pop()`	Removes a random element
Clear set	`my_set.clear()`	Removes all items
Union	`a.union(b)` or `a | b`	Common, Uncommon items
Intersection	`a.intersection(b)` or `a & b`	Common items
Difference	`a.difference(b)` or `a - b`	Items in `a` but not in `b`

Add item

➔ To add a new element to a set, use the `.add()` method.

Syntax:

```
set_name.add(item)
```

Example code:	Output:

`fruits = {"apple", "banana"}`
`fruits.add("cherry")`
 	
`print(fruits)`
	`{'apple', 'banana', 'cherry'}`
	
	`{'cherry', 'apple', 'banana'}`

Notes:

- If the item already exists, it **won't be added again** (sets store unique values only).
- Order is **not guaranteed** when printing a set.

Remove item

➔ Python provides **two main ways** to remove items from a set: `.remove()` and `.discard()`.

Syntax: Using `.remove()`

```
set_name.remove (item)
```

Example code:

```
fruits = {"apple", "banana", "cherry"}
fruits.remove("banana")

print(fruits)
```

Output:

```
{'cherry', 'apple'}
```

Notes:

- Removes a specific element.
- Raises an **error** if the element is not found.

Discard item

Syntax: Using `.discard()`

```
set_name. discard(item)
```

Example code:

```
fruits = {"apple", "banana", "cherry"}
fruits.discard("banana")
fruits.discard("orange")  # No error!

print(fruits)
```

Output:

```
{'cherry', 'apple'}
```

Notes:

- Also removes a specific element.
- **Does NOT raise an error** if the item is missing.

Pop random item

➔ The `.pop()` method **removes and returns a random element** from a set because sets are unordered.

Syntax:

```
set_name.pop()
```

Example code:

```
colors = {"red", "green", "blue"}
removed_item = colors.pop()

print("Removed:", removed_item)
print("Remaining:", colors)
```

Output:

```
Removed: blue
Remaining: {'green', 'red'}
```

Notes: Raises **`KeyError`** if the set is **empty**.

Union sets

➔ The **union** of two or more sets returns a new set containing **all unique elements** from the sets involved.

Syntax:

```
set1.union(set2, set3, ...)
# or
set4 = set1 | set2 | set3
```

Example code:

```
a = {"apple", "banana"}
b = {"banana", "cherry"}
c = {"Mango"}
d = a.union(b)
e = a | b | c

print(d)
print(e)
```

Output:

```
{'cherry', 'banana', 'apple'}
{'cherry', 'banana', 'Mango', 'apple'}
```

Notes:

- Duplicates are automatically removed.
- Original sets remain unchanged.

Intersection sets

➔ The **intersection** of sets returns a new set containing **only elements common** to all sets.

Syntax:

```
set1.intersection(set2, set3, ...)
# or
set4 = set1 & set2 & set3
```

Example code:

```
a = {"apple", "banana", "cherry"}
b = {"banana", "cherry", "grape"}
c = {"Mango", "cherry", "grape"}

d = a.intersection(b)
e = a & b & c

print(d)
print(e)
```

Output:

```
{'cherry', 'banana'}
{'cherry'}
```

Notes:

- Does **not modify** the original sets.
- Result contains **only shared elements**.

Clear set

➔ To remove **all elements** from a set, use the `.clear()` method.

Syntax:

```
set_name.clear()
```

Example code:

```
fruits = {"apple", "banana", "cherry"}
fruits.clear()

print(fruits)
```

Output:

```
set()
```

Intersection sets

➔ The **difference** returns a new set containing elements that are **only in the first set** and **not in the others**.

Syntax:

```
set1.difference(set2, set3, ...)
# or
set4 = set1 - set2 - set3
```

Example code:

```
a = {"apple", "banana", "cherry"}
b = {"banana", "grape"}
c = {"Mango", "cherry", "grape"}

d = a.difference(b)
e = a - b - c

print(d)
print(e)
```

Output:

```
{'apple', 'cherry'}
{'apple'}
```

Notes:

- The original sets are not changed.
- Order doesn't matter in sets, but the **set from which we subtract comes first.**

Practice Problem

Problem1: You are given a list of subjects chosen by students. Each **unique subject** needs **one classroom**. **Write a Python program** to calculate **how many classrooms** are needed.
["python", "java", "C++", "python", "javascript", "java", "python", "java", "C++", "C"]

Code:

```
subjects = ["python", "java", "C++", "python",
"javascript","java", "python", "java", "C++", "C"]

unique_subjects = set(subjects)
classrooms_needed = len(unique_subjects)

print("Classrooms needed:", classrooms_needed)
```

Output:

```
Classrooms needed: 5
```

Problem2: Figure out a way to **store both 9 and 9.0 as separate values** in a Python set.

Usually, `9` (int) and `9.0` (float) are considered **equal in value**, so a set will treat them as the same.

Example code:

```
set = {9, 9.0}
print(set)
```

Output:

```
{9}
```

Why this happens: Because,

```
9 == 9.0  # True
type(9) != type(9.0)  # True, but doesn't matter in set
```

Solution: Using string and int

Example code:

```
my_set = {9, "9.0"}
print(my_set)
```

Output:

```
{9, '9.0'}
```

Better Solution: Store with type info using tuple

Example code:

```
my_set = {(9, int), (9.0, float)}
print(my_set)
```

Output:

```
{(9, <class 'int'>), (9.0, <class 'float'>)}
```

Functions

➔ A **function** is a block of reusable code that performs a specific task.

Define a Function:	Call a Function:
`def function_name():` `    # code block`	`function_name()`
Example code:	**Output:**

`def greet():`
`    print("Hello, welcome to Python!")`
 	
`greet()`
	`Hello, welcome to Python!`

Notes: Use `def` to define a function.	

Function with Parameters

Define a Function:	Call a Function:
`def function_name(Parameters):` `    # code block`	`function_name(Parameters)`
Example code:	**Output:**

`def greet(name):`
`    print("Hello", name)`
 	
`greet("Mohsin")`
`greet("Jam")`
`greet("Alif")`
`greet("Nihan")`
	`Hello Mohsin`
	`Hello Jam`
	`Hello Alif`
	`Hello Nihan`

Notes: Functions help make code **modular** and **reusable**.	

Function with Return Value:

Example code:	Output:
```def add(a, b): return a + b result = add(3, 5) print(result)```	**8**

Notes: `return` sends a value back to the caller.

Practice Problem

Problem1: Write a function in Python that takes a list as a parameter and **prints its length**.

Code:	Output:
```def print_list_length(lst): print("Length of the list:", len(lst)) my_list = [10, 20, 30, 40] print_list_length(my_list)```	**Length of the list: 4**

Problem2: Write a function that takes a list as a parameter and **prints all its elements** in a single line.

Code:	Output:

``def print_elements_single_line(lst):
    for item in lst:
        print(item, end=' ')
    print()
 	
my_list = [1, 2, 3, 4, 5]
print_elements_single_line(my_list)```
	1 2 3 4 5

Problem3: Write a function in Python that **takes a number** `n` as a parameter and **prints its factorial.**

Code:	Output:

``
def print_factorial(n):
    factorial = 1
    for i in range(1, n + 1):
        factorial *= i
    print("Factorial of", n, "is", factorial)
 	
num = int(input("Enter a Number: "))
 	
print_factorial(num)
``
	Enter a Number: 5
	Factorial of 5 is 120

Problem4: Write a function in Python that **takes an amount in USD** and **converts it to BDT.** Assume 1 USD = 117 BDT.

Code:	Output:

``
def convert_usd_to_bdt(usd):
    bdt = usd * 117
    print(f"{usd} USD = {bdt} BDT")
 	
amount = int(input("Enter Amount: "))
 	
convert_usd_to_bdt(amount)
``
	Enter Amount: 50
	50 USD = 5850 BDT

Recursion

➔ **Recursion** is a method of solving problems where a function calls itself.

Structure of Recursive Function:

```
def function_name():
    if base_condition:
        return result
    else:
        return function_name()  # recursive call
```

Example: Factorial Using Recursion

```
def factorial(n):
    if n == 0 or n == 1:
        return 1  # base case
    else:
        return n * factorial(n - 1)  # recursive case

print(factorial(5))  # Output: 120
```

Output:

```
120
```

Key Points:

- Every recursive function must have a **base case** to stop the recursion.
- Without a base case, it will lead to **infinite recursion** (and crash with an error).
- Useful for problems like:
  - Factorial
  - Fibonacci
  - Tree traversal
  - Tower of Hanoi

Practice Problem

Problem1: Write a recursive function to calculate the sum of the first n natural numbers.

Code:

```
def sum_natural(n):
    if n == 1:
        return 1
    else:
        return n + sum_natural(n - 1)
num = int(input("Enter a Number: "))
print("Sum: ", sum_natural(num))
```

Output:

```
Enter a Number: 5
Sum:  15
```

OBJECT-ORIENTED PROGRAMMING (OOP)

Object-Oriented Programming (OOP)

➔ OOP is a programming paradigm that organizes code into **classes** and **objects**.

Key Concepts of OOP:

Concept	Description
Class	Blueprint for creating objects
Object	Instance of a class
Attribute	Variables inside a class
Method	Functions inside a class
Constructor	`__init__()` method called when object is created
Self	Refers to the current object

Basic OOP Example:

```
class Student:
    def __init__(self, name, age):
        self.name = name        # attribute
        self.age = age

    def greet(self):            # method
        print("Hello, my name is", self.name)
        print("And my age is", self.age)

# Create object
s1 = Student("Mohsin", 22)

# Access method
s1.greet()
```

Output:

```
Hello, my name is Mohsin
And my age is 22
```

🧠 OOP Features:

Feature	Example
Encapsulation	Hiding data (using private variables)
Inheritance	One class inherits from another
Polymorphism	Same method behaves differently in different classes
Abstraction	Hiding complex details using abstract classes (via `abc` module)

Encapsulation

➔ **Encapsulation** is the OOP principle of **hiding internal details** and protecting object data from direct modification.

How It Works:

- ⇨ **Private attributes**: Defined using **underscore _** or **double underscore __**.
- ⇨ Accessed using **getter and setter methods**.

Example code:

```
class Student:
    def __init__(self, name, age):
        self.__name = name   # private attribute
        self.__age = age

    def get_name(self):      # getter method
        return self.__name


    def set_name(self, name):  # setter method
        self.__name = name


# Create object
s1 = Student("Mohsin", 22)

print(s1.get_name())       # Output: Mohsin

s1.set_name("Alif")
print(s1.get_name())       # Output: Alif
```

Output:

```
Mohsin
Alif
```

Why Use Encapsulation?

- Prevents unauthorized access.
- Makes the class modular and secure.
- Allows validation before changing data.

Inheritance

➔ **Inheritance** allows a class (child) to **acquire properties and methods** from another class (parent), promoting code reusability.

Syntax:

```
class Parent:
    # Parent class content

class Child(Parent):
    # Child class inherits Parent
```

Example code:

```
class A:
    def methodA(self):
        print("A method")

class B:
    def methodB(self):
        print("B method")

class C(A, B):
    pass

obj = C()
obj.methodA()
obj.methodB()
```

Output:

```
A method
B method
```

🧠 Types of Inheritance in Python:

Type	Description
Single Inheritance	One child class inherits one parent
Multiple Inheritance	One class inherits from multiple parents
Multilevel Inheritance	Inheritance across multiple levels
Hierarchical Inheritance	Multiple child classes from one parent
Hybrid Inheritance	Combination of multiple types

super() Keyword

➔ The `super()` keyword is used to **access methods and constructors of a parent class** from the child class.

Why Use super()?

- ⇨ To reuse the parent class's **methods or __init__() constructor**.
- ⇨ Avoids hardcoding the parent class name.
- ⇨ Useful in **inheritance** (especially with multiple inheritance).

Example code:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
        print(f"Person: {self.name}, Age: {self.age}")

class Student(Person):
    def __init__(self, name, age, student_id):
        super().__init__(name, age)   # Call parent constructor with parameters
        self.student_id = student_id
        print(f"Student ID: {self.student_id}")

# Create object
s = Student("Mohsin", 20, "23-50194-1")
```

Output:

```
Person: Mohsin, Age: 20
Student ID: 23-50194-1
```

Summary:

- super() helps in calling **parent class methods or constructors.**
- Makes code **clean and maintainable**.
- Essential **in constructor chaining** and **method overriding.**

Single Inheritance

➔ **Single Inheritance** is when a **child class inherits** from **only one parent class**.

Syntax:

```
class Parent:
    # parent class content

class Child(Parent):
    # inherits from Parent
```

Example code:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age


class Student(Person):
    def __init__(self, name, age, student_id):
        super().__init__(name, age)
        self.student_id = student_id


# Create object
s1 = Student("Mohsin", 22, "23-50194-1")
print(s1.name)
print(s1.age)
print(s1.student_id)

s2 = Student("Jam", 20, "23-50187-1")
print(s2.name)
print(s2.age)
print(s2.student_id)
```

Output:

```
Mohsin
22
23-50194-1

Jam
20
23-50187-1
```

Key Points:

- Promotes code reusability
- Only one parent class
- The child can use all non-private members of the parent

Multiple Inheritance

➔ **Multiple Inheritance** is when a child class inherits from **more than one parent class**.

Syntax:

```
class Parent1:
    # code

class Parent2:
    # code

class Child(Parent1, Parent2):
    # inherits from both Parent1 and Parent2
```

Example code:

```
class Father:
    def skill(self):
        print("Father: Farming")

class Mother:
    def talent(self):
        print("Mother: Cooking")

class Child(Father, Mother):   # Multiple Inheritance
    def hobby(self):
        print("Child: Drawing")

c = Child()
c.skill()     # Inherited from Father
c.talent()    # Inherited from Mother
c.hobby()     # Own method
```

Output:

```
Father: Farming
Mother: Cooking
Child: Drawing
```

Key Points:

- Child class gets properties from **multiple parent classes**.
- Python uses **Method Resolution Order (MRO)** to decide which method to use if there's a conflict.
- Helps combine features from multiple sources.

Multilevel Inheritance

➔ **Multilevel Inheritance** means a class is derived from a **class that is already derived from another class**.
In simple words: **Child → Parent → Grandparent**

Syntax:

```
class Grandparent:
    # code

class Parent(Grandparent):
    # code

class Child(Parent):
    # inherits from Parent (and indirectly Grandparent)
```

Example code:

```
class Grandfather:
    def house(self):
        print("Grandfather: Owns a house")

class Father(Grandfather):
    def car(self):
        print("Father: Owns a car")

class Son(Father):
    def laptop(self):
        print("Son: Owns a laptop")

s = Son()
s.house()    # From Grandfather
s.car()      # From Father
s.laptop()   # From Son
```

Output:

```
Grandfather: Owns a house
Father: Owns a car
Son: Owns a laptop
```

Key Points:

- Inherits features in **a chain** (like 3 generations).
- Child has access to all **public members of Parent** and **Grandparent.**
- Helps build **stepwise structure** in OOP design.

Abstraction

➔ **Abstraction** means **hiding the internal implementation** and **showing only essential features** to the user.

In Python, abstraction is achieved using:

- **Abstract classes**
- **Abstract methods**

👉 Done using the `abc` module (`abc` = Abstract Base Class)

Syntax:

```
from abc import ABC, abstractmethod

class MyAbstractClass(ABC):
    @abstractmethod
    def abstract_method(self):
        pass
```

Example code:

```
from abc import ABC, abstractmethod

class Animal(ABC):
    @abstractmethod
    def sound(self):
        pass

class Dog(Animal):
    def sound(self):
        print("Dog: Woof!")

class Cat(Animal):
    def sound(self):
        print("Cat: Meow!")

# a = Animal()  ❌ Cannot create object of abstract class
d = Dog()
d.sound()

c = Cat()
c.sound()
```

Output:

```
Dog: Woof!
Cat: Meow!
```

Key Points:

- Abstract class cannot be instantiated.
- All child classes must override the abstract methods.
- Encourages design consistency.
- Promotes security and cleaner code.

Polymorphism

➔ **Polymorphism** means **"many forms"** — the ability of different classes to respond to the same method in different ways.

It allows:

- ⇨ Same method name to behave differently on different classes.
- ⇨ Flexibility in code design.

Example code:

```
class Cat:
    def sound(self):
        print("Cat: Meow")

class Dog:
    def sound(self):
        print("Dog: Woof")

# Polymorphism
def make_sound(animal):
    animal.sound()

c = Cat()
d = Dog()

make_sound(c)  # Cat: Meow
make_sound(d)  # Dog: Woof
```

Output:

```
Cat: Meow
Dog: Woof
```

Key Points:

- Allows same **interface, different behavior.**
- Achieved via **method overriding** or **common method** names across classes.
- Increases **code reusability and scalability**.

Static Methods

➔ A **static method** is a method that **does not take** `self` **or** `cls` as the first argument.
It does **not depend on class or instance state**.

Why Use Static Methods?

⇨ Used when some functionality is **logically related to the class**, but **doesn't need access to class or instance variables**.

Defined using:

```
@staticmethod
```

Example code:

```
class MathTools:
    @staticmethod
    def add(x, y):
        return x + y

    @staticmethod
    def multiply(x, y):
        return x * y

# Accessing without creating object
print(MathTools.add(5, 3))       # 8
print(MathTools.multiply(4, 6))  # 24

# Can also access via object
tool = MathTools()
print(tool.add(2, 2))            # 4
```

Output:

```
8
24
4
```

Key Points:

- Decorated with **@staticmethod**
- No **self** or **cls** parameter
- Can be called using class name or object
- Ideal for **utility/helper functions** related to a class

Class Methods

➔ A **class method** is a method that **works with the class itself**, not the instance. It's defined using the `@classmethod` decorator, and it takes `cls` (class) as the first parameter.

Syntax:

```
@classmethod
def method_name(cls):
    # code
```

Example code:

```
class Student:
    University_name = "AIUB"

    def __init__(self, name):
        self.name = name

    @classmethod
    def change_University(cls, new_name):
        cls.University_name = new_name

# Before changing
print(Student.University_name)  # AIUB

# Using class method
Student.change_University("BUET")

# After changing
print(Student.University_name)  # BUET
```

Output:

```
AIUB
BUET
```

Key Points:

- Uses `@classmethod`
- First argument is `cls` (refers to the class)
- Can **access or modify class variables**
- Can be called using **class name or instance**

Practice Problem

Problem1: Write a Python program to define a class named `BankAccount` that models a simple bank account. The class should have the following:

- **Attributes**:
  - `name` (the name of the account holder)
  - `balance` (the current account balance)
- **Methods**:
  - `deposit(amount)` – adds the specified amount to the balance
  - `withdraw(amount)` – deducts the amount from the balance if sufficient funds are available, otherwise display an error
  - `display_balance()` – displays the current balance of the account

Create an object of the class and demonstrate the use of each method.

Code:	Output:

```python
def sum_natural(n):
class BankAccount:
    def __init__(self, name, balance):
        self.name = name
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        print(f"{amount} BDT deposited successfully.")

    def withdraw(self, amount):
        if amount <= self.balance:
            self.balance -= amount
            print(f"{amount} BDT withdrawn successfully.")
        else:
            print("Insufficient balance!")

    def display_balance(self):
        print(f"Account Holder: {self.name}")
        print(f"Current Balance: {self.balance} BDT")


# Example usage
account1 = BankAccount("Mohsin", 10000)

account1.display_balance()
account1.deposit(2000)
account1.withdraw(5000)
account1.display_balance()
```

```
Account Holder: Mohsin
Current Balance: 10000 BDT
2000 BDT deposited successfully.
5000 BDT withdrawn successfully.
Account Holder: Mohsin
Current Balance: 7000 BDT
```

Problem2: Write a Python program to implement **inheritance** using object-oriented programming.

- Create a base class named `Shape` with a method `area()` that returns 0.
- Derive the following subclasses from `Shape`:
  - `Square` – with a constructor to accept `side` and override the `area()` method.
  - `Circle` – with a constructor to accept `radius` and override the `area()` method.
  - `Triangle` – with a constructor to accept `base` and `height` and override the `area()` method.

Demonstrate polymorphism by calling the `area()` method for each shape.

Code:	Output:

```python
import math

class Shape:
    def area(self):
        return 0

class Square(Shape):
    def __init__(self, side):
        self.side = side

    def area(self):
        return self.side * self.side

class Triangle(Shape):
    def __init__(self, base, height):
        self.base = base
        self.height = height

    def area(self):
        return 0.5 * self.base * self.height


# Create a list of different shapes
shapes = [Square(4), Triangle(6, 5)]

# Loop through each shape and print its name and area
print("Shape Areas:\n" + "-"*20)
for shape in shapes:
    name = shape.__class__.__name__  # Get the class name
    area = shape.area()              # Call the area method
    print(f"{name} area: {area:.2f}")
```

```
Shape Areas:
--------------------
Square area: 16.00
Triangle area: 15.00
```

Problem3: Write a Python program to demonstrate the concept of **inheritance** and the use of the `super()` function.

- Create a base class `Person` with attributes `name` and `age`.
- Create a derived class `Employee` that inherits from `Person` and adds `salary` and `department` attributes.
- Use the `super()` function to initialize the base class attributes from the derived class constructor.
- Display the details of the employee.

Code:	Output:

```python
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def display(self):
        print(f"Name: {self.name}")
        print(f"Age: {self.age}")

class Employee(Person):
    def __init__(self, name, age, salary, department):
        super().__init__(name, age)
        self.salary = salary
        self.department = department

    def display(self):
        super().display()
        print(f"Salary: {self.salary}")
        print(f"Department: {self.department}")

# Example usage
emp1 = Employee("Alif Hasan Khan", 25, 50000, "IT")
emp1.display()

print("-"*20)
emp2 = Employee("Mohsin Ibna Hossain", 27, 70000, "IT")
emp2.display()
```

```
Age: 25
Salary: 50000
Department: IT
--------------------
Name: Mohsin Ibna Hossain
Age: 27
Salary: 70000
Department: IT
```

Problem4: Design a Movie Ticket Booking System using OOP in Python:

- Create a base class `Ticket` with attributes: `_price` (private), `seat_no`.
- Include methods to **get the price**, **set the price**, and **display ticket info**.
- Create a derived class `StudentTicket` that inherits from `Ticket` and applies a discount to the ticket price.
- Demonstrate **encapsulation** (via private attributes and getter/setter methods) and use of **`super()`** to initialize base class attributes.

Code:	Output:

```python
class Ticket:
    def __init__(self, price, seat_no):
        self._price = price     # Encapsulated (private) attribute
        self.seat_no = seat_no

    def get_price(self):
        return self._price

    def set_price(self, price):
        if price >= 0:
            self._price = price
        else:
            print("Invalid price!")

    def display_info(self):
        print(f"Seat No: {self.seat_no}")
        print(f"Ticket Price: BDT {self._price}")

class StudentTicket(Ticket):
    def __init__(self, price, seat_no, discount):
        super().__init__(price, seat_no)
        self.discount = discount

    def apply_discount(self):
        discounted_price = self.get_price() * (1 -
self.discount / 100)
        self.set_price(discounted_price)

    def display_info(self):
        self.apply_discount()
        print("Student Ticket Info:")
        super().display_info()

ticket1 = StudentTicket(500, "A12", 20)
ticket1.display_info()
```

```
Student Ticket Info:
Seat No: A12
Ticket Price: BDT 400.0
```

FILE I/O

File I/O

➔ File I/O lets your Python program **read from** or **write to** files.

Key keyword of File I/O:

Mode	Description
`'r'`	Read (default)
`'w'`	Write (overwrite)
`'a'`	Append
`'x'`	Create
`'b'`	Binary mode
`'t'`	Text mode (default)
`'r+'`	Read and write

Open, read & close File:

Reading a File:

```
file = open("demo.txt", "r")
print(file.read())
file.close()
```

Writing to a File:

```
file = open("demo.txt", "w")
file.write("Hello, World!")
file.close()
```

Appending to a File:

```
file = open("demo.txt", "a")
file.write("\nAppended line.")
file.close()
```

Best Practice: Use `with` statement, automatically closes the file:

```
with open("demo.txt", "r") as file:
    data = file.read()
    print(data)
```

Common Methods:

Method	Description
`read()`	Reads entire content
`readline()`	Reads a single line
`readlines()`	Returns list of lines
`write(text)`	Writes string to file
`writelines(list)`	Writes list of strings to file
`close()`	Closes the file

Writing a File

➔ Overwrites the file **if it exists**. **Creates a new** file **if it doesn't.**

Syntax:

Method1: Using `"w"` mode (Write)

```
with open("example.txt", "w") as file:
    file.write("This is the first line.\n")
    file.write("This is the second line.\n")
```

Notes:

- **\n** adds a new line.
- Using with is safer as it auto-closes the file.

Reading a File

Syntax:

Method1: Using `open()` and `read()`

```
file = open("example.txt", "r")
content = file.read()
print(content)
file.close()

#reads entire file as a single string.
```

Method2: Using `with` (Best Practice)

```
with open("example.txt", "r") as file:
    content = file.read()
    print(content)

#Automatically closes the file after reading.
```

Notes:

- File must exist, or you'll get a **FileNotFoundError.**
- Always close the file using **file.close()** unless you use **with.**

Deleting a File

➔ To delete a file, you use the `os` module.

Syntax:

Step1: Import `os`

```
import os
```

Step2: Delete a File using `os.remove()`

```
import os

os.remove("example.txt")
```

Best Practice: Check if File Exists First

```
import os

if os.path.exists("example.txt"):
    os.remove("example.txt")
    print("File deleted successfully.")
else:
    print("File does not exist.")
```

Notes: If the file doesn't exist and you don't check, it will raise a `FileNotFoundError`.

Practice Problem

Problem1: Write a Python program to perform the following operations using **File I/O**:

1. Create a new file named `practice.txt` and write the following content in it:

```
Hi everyone
we are learning File I/O
using Java.
I like programming in Java.
```

2. Write a function to **replace all occurrences** of the word `"Java"` with `"Python"` in the file.
3. Search whether the word `"learning"` exists in the file or not, and print the result.

Code:	Output:

```
# Step 1: Create and write data to the file
with open("practice.txt", "w") as f:
    f.write("Hi everyone\n")
    f.write("we are learning File I/O\n")
    f.write("using Java.\n")
    f.write("I like programming in Java.\n")

# Step 2: Function to replace 'Java' with 'Python'
def replace_java_with_python():
    with open("practice.txt", "r") as f:
        data = f.read()

    data = data.replace("Java", "Python")

    with open("practice.txt", "w") as f:
        f.write(data)
```

```
The word 'learning' exists in the file.
```

```python
# Step 3: Function to search for 'learning'
def search_learning():
    with open("practice.txt", "r") as f:
        content = f.read()
        if "learning" in content:
            print("The word 'learning' exists in the file.")
        else:
            print("The word 'learning' does not exist in the file.")

# Run the functions
replace_java_with_python()
search_learning()
```

Problem2:

Write a Python function to **find the line number** where the word `"learning"` occurs **first** in a given file.

- Return the **line number** (starting from 1).
- Return **-1** if the word does not exist in the file.

Code:	Output:

`def find_learning_line(filename):`
`  with open(filename, "r") as file:`
`  for line_number, line in enumerate(file, start=1):`
`  if "learning" in line:`
`  return line_number`
`  return -1`
 	
`# Example usage:`
`filename = "practice.txt"`
`result = find_learning_line(filename)`
`print("Line number:", result)`
	
	`Line number: 2`

IMPORT IN PYTHON

import in Python

➔ Importing an Entire Module

Importing an Entire Module:

```
import math
print(math.sqrt(16))
# Output: 4.0
```

Import with Alias:

```
import math as m
print(m.pi)
# Output: 3.141592653589793
```

Import Specific Function or Variable:

```
from math import sqrt, pi
print(sqrt(25))  # Output: 5.0
print(pi)
# Output: 3.141592653589793
```

Import All Functions (Not Recommended):

```
from math import *
print(sin(90))
```

Common Modules:

Module	Purpose
`math`	Math operations
`random`	Random number generation
`datetime`	Date and time handling
`os`	File system operations
`sys`	System-specific functions

random Modules

random integer between **a** and **b** (inclusive):

```
import random
print(random.randint(1, 10))
# Output: Any integer from 1 to 10
```

random element from a **list**, **tuple**, or **string**:

```
import random
print(random.uniform(1, 5))
# Example: 2.39485
```

Shuffles the list **in place** (modifies original list):

```
import random
cards = [1, 2, 3, 4, 5]
random.shuffle(cards)
print(cards)
# Output: e.g. [3, 5, 1, 2, 4]
```

random float between **a** and **b**:

```
import random
colors = ["red", "green", "blue"]
print(random.choice(colors))
 # Output: "green", etc.
```

CREATING A VIRTUAL ENVIRONMENT

What is a Virtual Environment?

⇨ A virtual environment is an **isolated Python workspace** that allows you to install and manage project-specific libraries without affecting the global Python installation.

❖ Why Use a Virtual Environment?

- Prevents dependency conflicts between projects
- Keeps packages project-specific
- Makes projects portable and reproducible
- Avoids permission issues from global installs

Creating a Virtual Environment

Step 1: Navigate to Project Directory

Command:	Output:
`cd project_directory`	PROBLEMS OUTPUT PORTS DEBUG CONSOLE TERMINAL PS C:\Users\ROWTECH\Desktop\Python\Lab> cd Lab1 PS C:\Users\ROWTECH\Desktop\Python\Lab\Lab1>

Step 2: Create the Virtual Environment

Command:	Output:
`python -m venv env_name`	PS C:\Users\ROWTECH\Desktop\Python\Lab\Lab1> python -m venv my_venv PS C:\Users\ROWTECH\Desktop\Python\Lab\Lab1> ls Directory: C:\Users\ROWTECH\Desktop\Python\Lab\Lab1 Mode LastWriteTime Length Name ---- ------------- ------ ---- d----- 2/9/2026 10:40 PM my_venv -a---- 2/9/2026 8:59 PM 55 Git LInk.txt

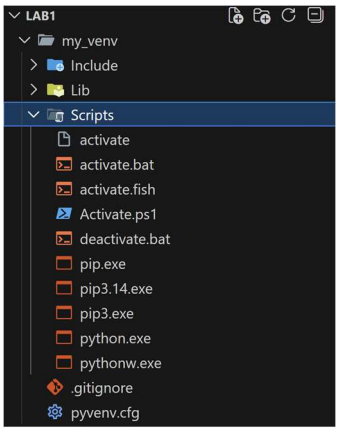
- ❖ python-m venv → invokes the built-in venv module
- ❖ env_name → user-defined environment name (e.g., venv, env, myenv)

Virtual Environment Directory Structure

➢ **After creation, the environment contains:**

- Scripts/ (Windows) or bin/ (Linux/macOS) → activation scripts
- Lib/ → installed site-packages
- pyvenv.cfg → Python environment configuration

❖ **Example:**



Activating the Virtual Environment

Windows:	
Command:	**Output:**
`env_name\Scripts\activate`	`PS C:\Users\ROWTECH\Desktop\Python\Lab\Lab1> my_venv\Scripts\activate` `(my_venv) PS C:\Users\ROWTECH\Desktop\Python\Lab\Lab1>`
Linux / macOS	
Command:	
`source env_name/bin/activate`	

❖ **Activation Indicator:**

➢ The command prompt shows: (env_name)

Package Management Inside Virtual Environment

- **Install a Package?**

Command:	Example:
`pip install package_name`	`PS C:\Users\ROWTECH\Desktop\Python\Lab\Lab1> my_venv\Scripts\activate` `(my_venv) PS C:\Users\ROWTECH\Desktop\Python\Lab\Lab1> pip install numpy`

- **View Installed Packages**

Command:	Example:

`pip list`	`(my_venv) PS C:\Users\ROWTECH\Desktop\Python\Lab\Lab1> pip list`
	`Package Version`
	`------- -------`
	`numpy 2.4.2`
	`pip 26.0.1`
	`(my_venv) PS C:\Users\ROWTECH\Desktop\Python\Lab\Lab1>`

- **Freeze Dependencies**

Command:	Example:

`pip freeze > requirements.txt`	`(my_venv) PS C:\Users\ROWTECH\Desktop\Python\Lab\Lab1> pip freeze > requirements.txt`
	`(my_venv) PS C:\Users\ROWTECH\Desktop\Python\Lab\Lab1> cat requirements.txt`
	`numpy==2.4.2`
	`(my_venv) PS C:\Users\ROWTECH\Desktop\Python\Lab\Lab1>`

Using requirements.txt

- **To install all dependencies on another system:** This ensures identical project environments.

Command:
`pip install -r requirements.txt`

Using requirements.txt

- **This returns the shell to the global Python environment.**

Command:	Example:
`Deactivate`	`(my_venv) PS C:\Users\ROWTECH\Desktop\Python\Lab\Lab1> deactivate` `PS C:\Users\ROWTECH\Desktop\Python\Lab\Lab1>`